NEXUS — Migration Integrity / Целостность миграций

RU · Что это и зачем

`schema_migrations` хранит, что запускалось, но не **что должно было запуститься**. Этого достаточно, чтобы не выполнить файл дважды, и недостаточно, чтобы заметить три вещи: что файл изменили; что номер переиспользовали; что шаг в середине не выполнялся вообще.

Практическое последствие на этом репозитории — не гипотетическое: `main` переписал уже применённую `0011_nexus_files.sql` на месте (+131/−8), превратив её из «создать `nexus_files`» в «создать `nexus_files` + `nexus_workspaces` + `nexus_workspace_files` + `nexus_pending_actions` + `nexus_voice_notes` + 10 `ALTER TABLE`». Старый раннер при несовпадении checksum выполняет файл заново. В результате:

- база, созданная старым `main`, поднимается, но строка `schema_migrations` перезаписывается новым checksum — **история перестаёт быть доказательством** того, что применялось;
- база линии PR17 (`0011_file_lab.sql`, таблицы `file_lab_*`) не стартует: `UndefinedColumn: column "name" of relation "nexus_files" does not exist`.

При включённом `AUTO_MIGRATE` это падение деплоя.

Оба сценария измерены на реальных базах, а не предположены.

EN · What this is and why

`schema_migrations` records what ran, not what should have run — enough to avoid applying a file twice, not enough to notice that a file changed, a number was reused, or a step never happened. That is not hypothetical here: `main` rewrote the already-applied `0011_nexus_files.sql` in place, and the old runner re-executes a file whose checksum changed. On a database from old `main` the history is silently overwritten; on a PR17-lineage database the service does not start. Both measured.

RU · Что проверяется

Pre-flight, до любого SQL:

Проверка	Что ловит
`files` в манифесте ⇄ файлы на диске	неотревьюенная миграция; потерянный шаг
уникальность номеров	переиспользованный номер (случай `0011`)
имя файла = `NNNN_name.sql`	порядок, зависящий от сортировки
история = префикс этого дерева	база из другой линии
checksum совпадает	изменённый уже применённый файл

Post-conditions, внутри транзакции файла:

Эффект	Проверка
таблицы и колонки	`information_schema.columns`
индексы	`pg_indexes`
RLS	`pg_class.relrowsecurity`

Индексы и RLS добавлены осознанно. Эксклюзивность задач держится на `uq_tasks_workspace_active` (миграция `0015`); раннер, который проверяет только таблицы, назвал бы базу здоровой, не имея этого инварианта. А `0016` существует ровно для того, чтобы ключ песочницы нельзя было прочитать через PostgREST, и не создаёт ни таблиц, ни колонок — без проверки RLS она была бы непроверяемой.

EN · What is verified

Pre-flight before any SQL: manifest ⇄ disk, unique numbers, filename shape, history as a prefix, checksum match. Post-conditions inside the file's transaction: tables, columns, indexes (`pg_indexes`), RLS (`pg_class.relrowsecurity`). Indexes and RLS are checked because P0's exclusivity rests on `0015`'s index, and `0016` only flips RLS flags — a tables-only check would call such a database healthy.

RU · Доказательство

`tests/test_migration_runner.py` — 20 тестов, каждый в собственной схеме:

№	Случай	Тест
1	эффект отсутствует	`test_a_migration_whose_effect_is_absent_is_not_recorded`
2	падение видно и не записано	`test_a_failing_migration_is_visible_not_recorded_and_blocks_later_ones`
3	зависимые не запускаются	там же
3b	файл откатывается целиком	`test_a_partial_file_rolls_back_completely`
4	имя файла в ошибке	`test_...blocks_later_ones`
4b	рестарт после сбоя	`test_a_restart_after_a_failure_resumes_at_the_broken_file`
5	изменённый файл	`test_editing_an_applied_migration_is_refused_not_re_executed`
5b	неотревьюенный файл	`test_an_unlisted_migration_file_is_refused`
5c	потерянный файл	`test_a_listed_but_missing_migration_is_refused`
6	дубль номера	`test_a_duplicate_migration_number_is_refused`
6b	плохое имя	`test_a_bad_filename_is_refused`

№	Случай	Тест
7	пропуск в истории	`test_history_that_skips_a_step_is_refused`
7b	чужая линия	`test_an_unknown_recorded_migration_is_refused`
8	чистый bootstrap	`test_a_clean_bootstrap_applies_the_real_migrations_then_is_a_no_op`
9	апгрейд	`test_upgrading_a_main_schema_database_converges`
10	манифест ⇄ SQL	`test_the_manifest_describes_the_files_it_is_shipped_with`
11	индекс отсутствует	`test_a_missing_index_is_caught`
12	индекс на месте	`test_a_declared_index_is_satisfied_when_it_exists`
13	RLS не включён	`test_rls_not_enabled_is_caught`
14	реальные 0015/0016	`test_real_index_and_rls_effects_are_verified_on_bootstrap`

На реальном дереве `main` (18 миграций, `0001…0020`): 20 passed.

Проверено вручную на чистой базе: bootstrap → `verify_schema: []`; повторный запуск → `[]`; `uq_tasks_workspace_active` существует; `cloud_sandboxes` и `conversation_routes` имеют `relrowsecurity = true`.

EN · Proof

20 tests in `tests/test_migration_runner.py`, each in a private schema, covering the 10 audit cases plus the new index and RLS cases. On the real 18-migration tree: 20 passed. Verified by hand on a clean database: bootstrap → `verify_schema: []`; re-run → `[]`; the P0 index exists; both secret tables have RLS enabled.

RU · Изменение поведения, которое надо согласовать

`AUTO_MIGRATE=true` — значение по умолчанию и то, на что рассчитывает `render.yaml`. Теперь запуск идёт через pre-flight и проверку эффектов. Значит:

> база, чья история не совпадает с манифестом, откажется стартовать, вместо того чтобы тихо «догнаться».

Для проверки целостности это правильный режим отказа, но **это изменение поведения деплоя**, и его надо пересмотреть до следующего релиза. Перед деплоем нужен read-only аудит прод-`schema_migrations` — он покажет, какая линия реально живёт в проде и не откажется ли стартовать этот сервис.

EN · Behaviour change to review

`AUTO_MIGRATE` now runs through pre-flight and effect verification, so a database whose history does not match the manifest refuses to start instead of silently

catching up. Correct for an integrity check, but it is a deploy behaviour change and must be reviewed before the next release, together with a read-only audit of the production `schema_migrations`.

RU · Ограничения

Манифест описывает дерево, а не живую базу. Базу, миграции которой применяли руками, он может законно отвергнуть — это требует аудита, а не правки манифеста.

Проверяются таблицы, колонки, индексы и флаг RLS. Не проверяются: политики RLS (их содержимое), гранты, внешние ключи, типы колонок, значения по умолчанию.

Отката (downgrade) нет — только восстановление из бэкапа.

Парсер эффектов — эвристика по SQL. Он дважды ошибся при написании (принял `length(...)` из `CHECK` за колонку и запятую внутри комментария за разделитель колонок), и оба раза это поймал bootstrap-прогон. **Ложное срабатывание блокирует корректную миграцию** — это цена строгости, и её надо знать. Манифест стоит перечитывать глазами при добавлении миграции.

EN · Limitations

The manifest describes the tree, not the live database (a hand-migrated database may legitimately be refused — that needs an audit, not a manifest edit). Verified: tables, columns, indexes, the RLS flag. Not verified: RLS policies, grants, foreign keys, column types, defaults. No downgrade path. The effect parser is a heuristic and was wrong twice while being written — both caught by the bootstrap run; a false positive blocks a correct migration, which is the price of strictness.